

Lab0_GitLab

1

- 有过。但并没有使用git，而是单纯在微信群聊里更新上传压缩包。
- 设计修改和暂存两个过程，可以在真正修改和提交之前check本次真正确定保留的修改。留有更多检查斟酌的空间，方便对比不同改动，且可以让提交记录更加精简和清晰。
- 前者 `git branch` 默认只显示本地分支，`git branch -a` 中 `-a` 是 `-all` 的缩写，会同时显示本地分支和远程跟踪分支。

2

实验步骤与必要截图

建立个人仓库后我利用配置好的 SSH 将其下载到电脑，首先对 `main.c` 文件进行了修改，并完成第一次 `commit`。第一次 `commit` 流程如下：

- 利用 `git status` 检查状态
- 暂存 `main.c`
- `git diff --staged` 再次确认暂存内容
- 提交 `commit`
- 上传到 GitHub

接下来学习分支管理。流程如下：

- `git switch -c feature` 创建新分支
- 在新分支上修改并提交
- `git switch main` 回到 `main` 分支
- 在 `main` 分支修改同一行并上传

- `git merge feature` 合并 `feature` 分支，遇到合并冲突。

CSAPP_Lab0_GitLab_25803050113 > C main.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      <<<<<< HEAD (当前更改)
6          printf("Whale and WinterSolstice -main\n");
7      =====
8          printf("Whale and WinterSolstice 4EVER -feature\n");
9      >>>>>> feature (传入的更改)
10 }
```

问题

终端



bash - CSAPP_Lab0_GitLab_25803050113



```
whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab0_GitLab_25803050113$ git merge feature
sh origin main
● whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab0_GitLab_25803050113$ git branch --show-current
main
⊗ whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab0_GitLab_25803050113$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
❖ whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab0_GitLab_25803050113$
```

- 查看冲突状态并解决冲突。手动解决冲突，将冲突标记完整删除，保存并检查。

```
● whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab0_GitLab_25803050113$ git status
On branch main
Your branch is up to date with 'origin/main'.

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   main.c

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    main
    main.o

no changes added to commit (use "git add" and/or "git commit -a")
○ whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab0_GitLab_25803050113$
```

- 标记冲突已解决，合并提交。

```
● whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab0_GitLab_25803050113$ git status
On branch main
Your branch is up to date with 'origin/main'.

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

● whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab0_GitLab_25803050113$ git log --oneline --graph --decorate --all
* 7496bf3 (HEAD -> main, origin/main, origin/HEAD) Merge feature into main and resolve conflict
| \
| * ee310e5 (origin/feature, feature) Modify main.c on feature branch
| * 7001c5b Modify main.c on main branch
| /
* 6e67d09 Complete TODO in main.c
* d3aa0ff Initial commit
○ whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab0_GitLab_25803050113$
```

3

为什么要学习git?

前面说到，之前在和同学共同完成项目时，我们采取的是用微信群传压缩包这种原始的方法。为了避免混淆版本，我们需要不停地给文件加上各种各样的后缀，比如“new1”“new2”之类。这种后缀很容易混淆，并且无法直观地体现出不同版本的代码差异、功能差异。

学习git，无论是个人开发还是团队合作，增添功能、版本比较或是修改bug，都可以迅速找到不同版本的不同之处，进行修改。

另外，git graph 功能还可以梳理版本树状图，十分清晰明了！

Commit Message规范：

这篇文章介绍了在提交 Commit 时的规范，可以帮助我们每次提交更加简洁易读，进行修改时也更加方便高效。

https://www.ruanyifeng.com/blog/2016/01/commit_message_change_log.html

commit应该清晰易懂。格式化的commit message方便快速浏览：可以只看行首、过滤信息，还能直接生成change log。本文介绍了commit message目前使用最广的写法Angular规范。

规范简介

该规范将commit message分为三个部分：**Header**、**Body**、**Footer**。前者必需。

```
<type>(<scope>): <subject>
```

```
<body>
```

```
<footer>
```

Header

type

说明类别。只有一行，只允许使用七个标识（标粗的必定出现在change log）：

- **feat**：新功能
- **fix**：修bug
- docs：文档
- style：格式
- refactor：重构
- test：增加测试
- chore：构建过程/

scope

说明commit影响的范围。

subject

是commit目的的简短描述，不超50字符。动词开头、第一人称现在时、首字母小写、无句点。

Body

对commit的详细描述。可分多行。第一人称现在时，应说明变动动机和对比。

Footer

只用于两种情况。

不兼容变动

如果当前代码与上一个版本不兼容，则Footer部分以 **BREAKING CHANGE** 开头，后面是对变动的描述、以及变动理由和迁移方法。

关闭Issue

如果当前commit针对某个issue，那么可以在Footer部分关闭这个issue（也可以多个）。

Revert

如果当前commit用于撤销以前的commit，则必须以 `revert:` 开头，后面跟着被撤销Commit的Header。

实用工具

Commitizen

撰写合格 Commit message 的工具。可在git commit时生成符合格式的Commit message。

validate-commit-msg

用于检查 Node 项目的 Commit message 是否符合格式。

Change log生成脚本

Gitflow使用规范

介绍了Gitflow的关键分支：`master`、`develop`、`feature`、`release`、`hotfix`。

同时以流程图的形式展示了开发过程一次迭代中git操作的示例。由 `master` 出发，以 `develop` 为主，中间不断进行功能修改（`feature`）、bug修改（`release`、`hotfix`），不断修改完善后再合并回 `master`。

还介绍了关键分支间的关系，在开发过程中不同关键分支以怎样的顺序进行提交，共同完成发布新功能、修改bug等任务。

除此之外这篇文章还展示了Gitflow使用示例。